SecureGen / S4: Generation-Time Defenses for LLM-Written Smart Contracts

A plain-language walkthrough of the pipeline

AIContractCheck project

July 1, 2026

Abstract

This note explains, in accessible terms, what the SecureGen "S4" system does and why. The measurement study behind this project found that vulnerabilities in LLM-generated Solidity are *intrinsic*: you cannot remove them by rewording the prompt. S4's answer is to put a lightweight security "verifier" *inside* the model's generation loop — a tiny probe that reads the model's own internal state token-by-token and steers it away from vulnerable code as it writes. This document walks through the idea, the training pipeline, the inference pipeline, and the current experimental results.

1 The problem, in one paragraph

Developers increasingly paste LLM-generated Solidity straight into production, where it guards real money. Our 2,400-contract, 8-LLM study showed that a large fraction of that code is vulnerable, and — crucially — that "adversarial" prompts do *not* meaningfully change the vulnerability rate ($p \approx 0.997$). The danger is not a clever attacker's prompt; it is the ordinary developer who skips the security pipeline. Asking the model nicely ("write it securely") does not fix this. So the fix has to live *in the generation process itself*.

2 The five strategies

SecureGen is a modular harness where each defense is a pluggable *Strategy*, and every strategy is scored by the *same* analyzer (Slither) so comparisons are fair.

Strategy	Idea	Cost
S0 baseline	plain generation (control)	1 call
S1 prompt prefix	prepend a security instruction	1 call
S2 / S2b repair loop	generate → Slither → repair	several calls
S3 checkpoint	write a skeleton first, then fill	several calls
S4 inline probe	**steer decoding with a probe**	1 call, white-box

S2b already gives a clean, *large* improvement (repairing a contract against Slither feedback). But S2b needs a full compile-and-Slither pass per fix — seconds each, and impossible *while the model is still typing*. **S4 is the novel contribution**: it replaces the slow analyzer-in-the-loop with a fast probe that works mid-generation.

3 The core insight behind S4

An "open" model (one whose weights we run ourselves — here `Qwen2.5-Coder-7B`) exposes its internal *activations*: the vectors flowing through each layer as it generates. We ask a simple question: is vulnerability already visible in those activations *before* the code is even finished? If yes, a tiny classifier can read that signal cheaply and warn us in real time.

TEACHER (Slither, slow, offline) → labels
STUDENT (tiny probe on activations) → fast inline signal
VERIFIER (Slither again, on the final code) → fair evaluation

The probe is a cheap, differentiable stand-in for a slow static analyzer — and it can ride *inside* decoding, where Slither physically cannot. That is the whole trick.

4 Pipeline 1 — Training the probe (one-time)

```
Qwen corpus contracts          Slither/Semgrep labels
dataset/qwen/*.sol (237)       (line numbers parsed from dedup_key #L88-L94)
       |                              |
       v                              v
  +-------------------------------------------------+
  |  extract_activations.py                         |
  |  re-feed each contract through Qwen (on the GPU) |
  |  grab the layer-12 residual stream               |
  |  one vector per LINE (the line's final token)    |
  |  label = 1 if any tool flagged that line         |
  +----------------------+--------------------------+
                         v
            activations.pt   { X:[N,3584]  y:[N]  groups:[N] }
                         v
  +-------------------------------------------------+
  |  train_probe.py -- tiny LINEAR classifier        |
  |  CONTRACT-LEVEL 80/20 split (no line leakage)    |
  +----------------------+--------------------------+
                         v
             probe_l12.pt   +   AUROC  <-- the paper result
```

5 Pipeline 2 — Steered decoding (inference)

```
prompt --> Qwen decodes token by token (with a KV cache for speed)
               |
               v   every ~16 tokens, at a ';' or '}' boundary:
       read layer-12 hidden state --> probe --> risk in [0,1]
               |
       risk > threshold?
          | no                      | yes
          v                         v
       keep going            BACKTRACK to the last safe point,
                             inject "// checks-effects-interactions..."
                             and re-decode the risky span
               |
```

```
            v
      final contract --> Slither (the harness verifies it, fairly)
```

Two engineering details make this runnable and correct:

- **KV cache**: without it, every new token re-reads the whole sequence ($O(n^2)$); with it, cost is $O(n)$. Verified: cache-on produces byte-identical output to cache-off.
- **Backtrack sync**: when we rewind the text, we crop the cache to the same point, so the model never "remembers" code we erased.

6 How we test "does it work?"

For each prompt we run the *same* white-box model twice with the *same* random seed:

- **OFF** — probe disabled: plain Qwen (the control).
- **ON** — probe active: steering + backtracking.

Because the seed matches, the two runs are identical until the probe first intervenes, so any difference in Slither's verdict is caused by *steering alone*. We then report the **paired Δvulns**: the per-prompt change ($\text{vulns}_{\text{ON}} - \text{vulns}_{\text{OFF}}$), averaged over prompts where both versions compiled. **Negative = steering removed vulnerabilities.** Pairing (each contract vs. itself) cancels the noise from contracts having different inherent difficulty.

7 Results so far

Probe (the signal exists)

On the RTX 3090, re-feeding 237 compiled Qwen contracts gave 10,287 line samples (13% flagged). A *linear* probe recovers Slither's judgment with:

	AUROC	**note**
Layer 12 (chosen)	0.887	lowest variance; mid-network
Best over sweep (L28)	0.887	signal is flat across depth
Range across all layers	0.86–0.89	robust: decodable everywhere

Conclusion: vulnerability is *linearly* decodable from Qwen's own layer-12 representations. This is the green light for steering — the signal the decoder needs is really there.

Steered decoding (does steering help?)

A paired A/B over 15 diverse prompts (one per contract category), threshold 0.5, gives a **mixed** result:

Metric (OFF → ON)	**Value**
Compile rate	**40% → 60%** (rescued 3 contracts)
Mean vulns (compiled only)	0.83 → 1.89
Density /100 LOC	1.25 → 3.25
High-severity total	0 → 3
Paired Δvulns ($n = 6$ both compiled)	**+0.33**

Honest reading: the probe signal is strong, and steering *improves compilability* (the nudge helps the model finish valid code). But it does *not* reduce vulnerability density: on contracts that compiled both ways vulns were flat except one regression, and the rescued contracts compile *with* vulnerabilities. **Conclusion: the signal works; naive comment-injection steering does not yet reduce vulns.** This motivates the next steps below (calibration via self-generation, a stronger steering intervention, and a threshold sweep) — and it is a genuine, publishable negative on the naive approach, not a dead end.

8 Status and what remains

Done	Next
Probe de-risk (AUROC 0.89)	Self-generation rewrite (clean number)
Layer sweep → layer 12	Scale A/B to all 300 prompts
KV-cache decoder (verified)	Threshold sweep for best trade-off
Paired A/B harness	Bigger open models (needs the A6000)

One-line summary: training time turns Slither's slow verdicts into a fast probe; inference time lets that probe steer the model away from vulnerable code as it writes — and the probe is accurate enough (AUROC ≈ 0.89) for that steering to be well-founded.